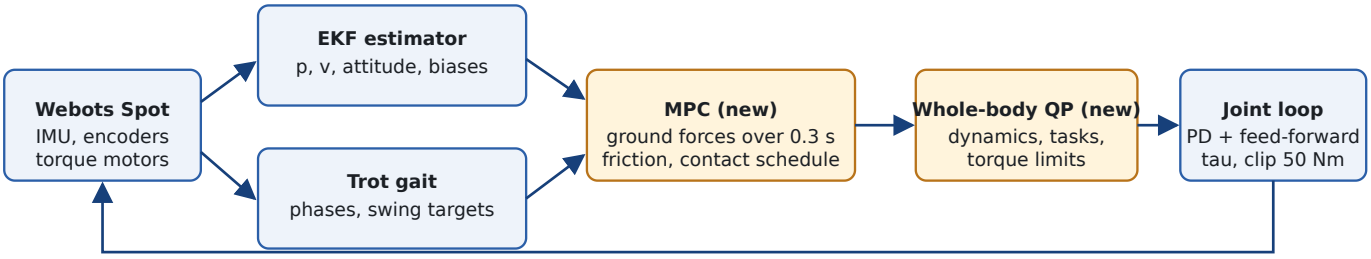
MPC and Whole-Body Control for Spot

What was built, how it fits the existing architecture, and how it performed in Webots (ROS 2 Jazzy, Python). All numbers are from simulation.

1. Where the controller sits



The controller runs **inside the Webots plugin, in the same 8 ms simulation step as the sensors** (no communication delay). Each step: the EKF updates the state, the gait advances, the MPC plans forces, the whole-body QP turns them into joint torques, and the joint loop applies `tau = Kp(q*-q) + Kd(dq*-dq) + tau_ff`. Three controllers share everything except how stance forces and torques are produced, selected by `controller:=`: `vmc` (spring-damper wrench + least-squares force split, the earlier controller), `mpc` (MPC forces, torques $-J^T R^T f$) and `mpc_wbc` (MPC forces + whole-body QP torques). If the MPC or the QP fails on a step, that step falls back to the simpler path.

2. Convex MPC (ground reaction forces)

A single rigid body driven by the foot forces f_i (state: attitude, position, angular and linear velocity, plus a constant gravity entry; 13 states):

$$\dot{\Theta} = R_z(\psi)^\top \omega, \qquad \dot{p} = v, \qquad \dot{\omega} = I_w^{-1} \sum_i r_i \times f_i, \qquad \dot{v} = \tfrac{1}{m} \sum_i f_i + g$$

r_i is the foot position relative to the centre of mass. The model is discretised (dt = 30 ms, N = 10 steps) and the states are eliminated, leaving a QP in the forces:

$$\min_f \sum_k \|x_k - x_k^{ref}\|_Q^2 + \alpha \|f_k\|^2 \quad \text{s.t.} \quad |f_x|, |f_y| \le \mu f_z, \;\; 0 \le f_z \le 220\,\text{N}, \;\; f_{i,k} = 0 \text{ for swing feet}$$

- **Contact schedule** comes from the trot phases: a foot can push only while it is in stance at that step; the touchdown positions of feet that will land inside the horizon are predicted with the Raibert rule. Reference: commanded velocity and yaw rate while walking (with a small integral on the velocity), a held position when standing; height 0.50 m, level attitude.
- **Solver:** no QP library was installed, so a dense OSQP-style ADMM solver was written in NumPy (`qp_solver.py`, tested against KKT conditions). Swing-foot variables are removed, forces are scaled, the step size adapts: about 10-20 iterations and 10-20 ms per solve.
- **Centre of mass** and inertia come from a mass model of the robot (below) instead of a guess.

3. Whole-body controller (WBC)

Generalised velocity $u = [v,\ \omega,\ \dot{q}]$ (18), dynamics $M(q)\dot{u} + h(q,u) = S^\top \tau + J_c^\top F$. Mass model from the Webots PROTO files: body 14.4 kg with inertia, plus per leg a 2.13 kg shoulder block, 0.94 kg thigh and 0.14 kg calf as point masses (27.2 kg total). One QP per step over the accelerations $\dot{u}$ and the stance forces F:

$$\min\ w_F \|F - F_{mpc}\|^2 + w_s \|J_s \dot{u} + \dot{J}_s u - a_{swing}\|^2 + w_b \|\text{base attitude/height accel. errors}\|^2 + \epsilon \|\dot{u}\|^2$$

subject to: floating-base dynamics (6 equations), planted feet do not accelerate $J_c \dot{u} + \dot{J}_c u = 0$, friction pyramids ($\mu = 0.75$, a little wider than the MPC's), and $|\tau| \le 50$ N·m. The output is the inverse-dynamics torque $\tau = [M\dot{u} + h - J_c^\top F]_{joints}$, used as feed-forward. The swing-foot acceleration target is the arch acceleration plus a task-space PD. Feet with almost no planned load (lift-off or touchdown instants) are left free, otherwise the near-zero friction cone makes the QP infeasible.

4. Results (Webots, 8 ms step, flat test world, headless)

Pushes are forces applied at the body centre for 0.2 s (a 100 N push changes the velocity by about 0.7 m/s). Directions rotate through the four world axes. Each cell is one run.

Test	Force controller (vmc)	MPC	MPC + WBC
Walk on a circle, command 0.30 m/s: mean speed / max tilt	0.32 m/s / 1.5°	0.32 m/s / 1.6°	0.33 m/s / 2.3° (full world, 0 fallbacks)
Standing, peak tilt after pushes of 100 / 120 / 140 / 160 / 180 / 200 N	2.4 / 5.9 / 8.9 / 8.2 / 8.8 / 16.1°	1.5 / 4.5 / 4.1 / 6.9 / 4.2 / 15.2°	3.1 / 8.0 / 4.3 / 8.6 / 6.4 / 16.7°
Walking, peak tilt after pushes of 100 / 140 / 160 / 180 / 200 / 220 / 240 N	5.0 / 4.8 / 8.6 / 10.4 / 6.9 / 26.7 / 4.7°	4.7 / 4.1 / 7.4 / 10.0 / 5.5 / 25.8 / 2.5°	5.4 / 6.7 / 5.0 / 15.4 / 10.5 / 21.2 / 5.8°
Standing: largest push survived (fell at the next one)	200 N (fell at 240)	240 N (fell at 280)	200 N (fell at 240)
Walking: largest push survived (fell at the next one)	220 N; 240 N depends on direction	220 N; 240 N depends on direction	280 N (fell at 320)
Control loop cost per 8 ms step (loaded laptop)	12-15 ms	25-30 ms	35-65 ms

- **All three controllers took every push in the 100-200 N sequences without falling** (the earlier version fell above about 130 N in the video). The MPC gives the gentlest response to standing pushes (peak tilt 16-54 % smaller than the force controller at 120-180 N). The whole-body variant did not show a consistent gain; with one run per cell, differences under a few degrees, or between 240 and 280 N, are within trial-to-trial variation and depend on push direction.
- Offline checks: 47 automated tests (QP solver, MPC on a reduced rigid-body simulation, dynamics model, whole-body QP, every controller mode through a walking sequence).

5. Problems found and fixed

Symptom	Cause	Fix
MPC robot fell within 0.2 s (growing 14 Hz roll oscillation)	MPC solved at 31 Hz and held for four steps: 32 ms of delay in a stiff loop	Re-solve every simulation step
MPC solve 260 ms, QP not converging	NumPy multi-threading on tiny matrices; poor scaling; swing variables included	Single-threaded BLAS, scaled forces, removed swing variables, adaptive step: 10-20 ms
Large pushes not caught	Recovery steps capped at 0.10-0.12 m; a 0.75 m/s shove needs about 0.17 m	Step limits 0.18 m (inward limit kept)
MPC walked about 8 % slow, then 20 % fast with the first integral	No integral, then too aggressive	Small integral on the velocity reference (0.12 /s, limit 0.04 m/s)
Whole-body QP fell back on 3 % of steps	Near-zero friction cone at lift-off / touchdown	Leave unloaded feet free, wider cone, accept near-converged solutions: 0 fallbacks in tests
Whole-body QP slow (31 ms)	Finite-difference Jacobians	Closed-form Jacobians (9 ms)

6. Limits and next steps

- Simulation only, one trial per condition; push direction changes the outcome as much as the controller does, so a repeated, randomised campaign is needed before ranking the three.
- The whole-body model neglects leg link inertia and its body inertia is a box estimate; it adds computing cost (about 2-4× the force controller) without a consistent benefit in these tests. The MPC is the more convincing addition: gentler standing-push response and a higher standing limit (240 N against 200 N), at about twice the compute.
- Flat ground only; stairs need terrain-aware foot heights in the planner.
- Commands: `ros2 launch spot_controller spot_launch.py control:=torque controller:=mpc` (or `mpc_wbc`); details in `docs/LOCOMOTION_CONTROL.md`.